

Prompt Ladder Assignment – Frontend AI Engineering

Topic:

Improving a **frontend coding prompt** through five iterations one layer at a time.

Project theme: Todo App UI. Tech choice (simple): **React + Vite + CSS**

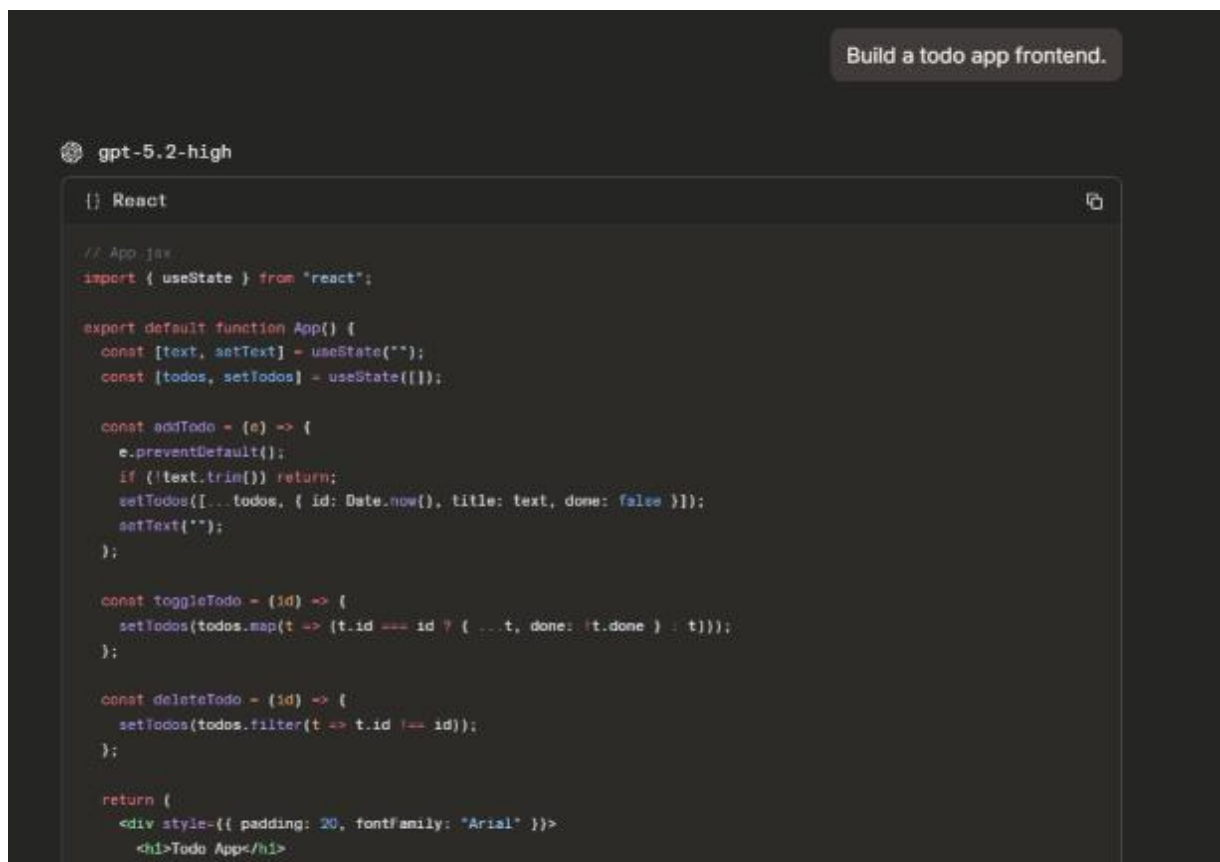
Baseline

Prompt:

“Build a todo app frontend.”

Representative Output:

- What changed in the prompt: None (baseline).
- What improved in the output: N/A
- What still failed: Too vague; unclear features, no structure, inconsistent code.
- What I would try next: Add a clearer goal.



The screenshot shows a chat interface with a dark background. At the top right, there is a grey button with the text "Build a todo app frontend." Below this, the model name "gpt-5.2-high" is displayed. The chat area shows a single message from the user, which is a code block for a React application. The code is for an "App.jsx" file and includes imports for "useState" from "react". It defines an "App" function with state for "text" and "todos". It also defines three functions: "addTodo" (which adds a new todo with a timestamp and title), "toggleTodo" (which toggles the "done" status of a todo), and "deleteTodo" (which filters out a todo by id). The "App" function returns a JSX element with a "div" containing a "h1" with the text "Todo App".

```
{} React

// App.jsx
import { useState } from "react";

export default function App() {
  const [text, setText] = useState("");
  const [todos, setTodos] = useState([]);

  const addTodo = (e) => {
    e.preventDefault();
    if (!text.trim()) return;
    setTodos([...todos, { id: Date.now(), title: text, done: false }]);
    setText("");
  };

  const toggleTodo = (id) => {
    setTodos(todos.map(t => (t.id === id ? { ...t, done: !t.done } : t)));
  };

  const deleteTodo = (id) => {
    setTodos(todos.filter(t => t.id !== id));
  };

  return (
    <div style={{ padding: 20, fontFamily: "Arial" }}>
      <h1>Todo App</h1>
    </div>
  );
}
```

```
return (  
  <div style={{ padding: 20, fontFamily: "Arial" }}>  
    <h1>Todo App</h1>  
  
    <form onSubmit={addTodo}>  
      <input  
        value={text}  
        onChange={(e) => setText(e.target.value)}  
        placeholder="New task..."  
      />  
      <button type="submit">Add</button>  
    </form>  
  
    <ul>  
      {todos.map((t) => (  
        <li key={t.id}>  
          <span  
            onClick={() => toggleTodo(t.id)}  
            style={{  
              cursor: "pointer",  
              textDecoration: t.done ? "line-through" : "none",  
            }}  
          >  
            {t.title}  
          </span>  
          <button onClick={() => deleteTodo(t.id)} style={{ marginLeft: 10 }}>  
            Delete  
          </button>  
        </li>  
      ))}  
    </ul>  
  </div>  
}
```

Version 1 — Layer: Clearer Goal

Prompt:

“Build a Todo app frontend in React with these features: add a task, list tasks, toggle complete, edit a task title, and delete a task.”

Representative Output:

- What changed in the prompt: Added a clearer goal (specific features).
- What improved in the output: The output included the missing UI actions, especially edit, Save, and Cancel, instead of only showing a basic todo list.
- What still failed: Still missing real project context (how to start, what files, what tooling).
- What I would try next: Add real context.

Version 1 Prompt:
“Build a Todo app frontend in React with these features: add a task, list tasks, toggle complete, edit a task title, and delete a task.”

 gpt-5.2-high

```
{ } React  
  
// src/App.jsx  
import { useState } from "react";  
  
export default function App() {  
  const [text, setText] = useState("");  
  const [todos, setTodos] = useState([]);  
  
  const [editingId, setEditingId] = useState(null);  
  const [editText, setEditText] = useState("");  
  
  const addTodo = (e) => {  
    e.preventDefault();  
    const title = text.trim();  
    if (!title) return;  
  
    setTodos((prev) => [  
      ...prev,  
      { id: Date.now(), title, done: false },  
    ]);  
  };  
}
```

```

    <button type="button" onClick={() => saveEdit(t.id)}>
      Save
    </button>
    <button type="button" onClick={cancelEdit}>
      Cancel
    </button>
  </div>
) : (
  <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
    <span
      onClick={() => toggleTodo(t.id)}
      style={{
        cursor: "pointer",
        textDecoration: t.done ? "line-through" : "none",
      }}
      title="Click to toggle complete"
    >
      {t.title}
    </span>
    <button type="button" onClick={() => startEdit(t)}>
      Edit
    </button>
    <button type="button" onClick={() => deleteTodo(t.id)}>
      Delete
    </button>
  </div>
)
  </li>
)
  </ul>
</div>
);
}

```

Version 2 — Layer: Real Context

Prompt:

“Build a Todo app frontend in React with these features: add, list, toggle complete, edit title, delete. This is for a beginner portfolio project using Vite + React, with no backend (use local component state only).”

Representative Output:

- What changed in the prompt: Added real context (beginner portfolio + Vite + no backend).
- What improved in the output: The output used local React component state and did not add backend code or API calls, so it matched a beginner Vite project.
- What still failed: Response can still be messy (no clear sections / files).
- What I would try next: Specify output format.

Version 2 Prompt :
 Build a Todo app frontend in React with these features: add a task, list tasks, toggle complete, edit a task title, and delete a task. This is for a beginner portfolio project using Vite + React, with no backend (use local component state only).

gpt-5.2-high

{} React
 📄

```

// src/App.jsx
import { useState } from "react";

export default function App() {
  // Portfolio friendly: keep everything local (no backend)
  const [text, setText] = useState("");
  const [todos, setTodos] = useState([
    { id: 1, title: "Learn React basics", done: false },
    { id: 2, title: "Build a small portfolio project", done: true },
  ]);

  const [editingId, setEditingId] = useState(null);
  const [editText, setEditText] = useState("");

  const addTodo = (e) => {
    e.preventDefault();
    const title = text.trim();
    if (!title) return;

    const newTodo = { id: Date.now(), title, done: false };

```

```

    </button>
    <button type="button" onClick={cancelEdit}>
      Cancel
    </button>
  </div>
</div>
) : (
  <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
    <span
      onClick={() => toggleTodo(t.id)}
      style={{
        cursor: "pointer",
        textDecoration: t.done ? "line-through" : "none",
      }}
      title="Click to toggle complete"
    >
      {t.title}
    </span>

    <button type="button" onClick={() => startEdit(t)}>
      Edit
    </button>

    <button type="button" onClick={() => deleteTodo(t.id)}>
      Delete
    </button>
  </div>
  )}
</li>
  )}
</ul>
</div>
);
}

```

Version 3 — Layer: Specified Output Format

Prompt:

“Build a Todo app frontend in React with these features: add, list, toggle complete, edit title, delete. This is for a beginner portfolio project using Vite + React, with no backend (use local state only). Organize your answer into exactly these sections: (1) File/Component Structure, (2) Code, (3) How to Run, (4) Manual Test Steps.”

Representative Output:

- What changed in the prompt: Added a specified output format (4 sections).
- What improved in the output: The output was organized into the exact 4 sections (File/Component Structure, Code, How to Run, Manual Test Steps), so it was easier to follow and copy/paste.
- What still failed : Even with sections, the response could still be verbose and it didn’t guarantee accessibility/responsive details unless explicitly required.
- What I would try next: Add quality criteria.

Version 3 Prompt:

Build a Todo app frontend in React with these features: add a task, list tasks, toggle complete, edit a task title, and delete a task. This is for a beginner portfolio project using Vite + React, with no backend (use local component state only).

Organize your answer into exactly these sections:

File/Component Structure
Code
How to Run
Manual Test Steps

gpt-5.2-high

1) File/Component Structure

For a beginner portfolio project, keep it simple:

```

{ } text

todo-prompt-ladder/
src/
  App.jsx
  main.jsx
  index.css

```

src/App.jsx

```
[ ] React

import { useState } from 'react';

export default function App() {
  const [text, setText] = useState("");
  const [todos, setTodos] = useState([
    { id: 1, title: 'Learn React basics', done: false },
    { id: 2, title: 'Build a portfolio project', done: true },
  ]);

  const [editingId, setEditingId] = useState(null);
  const [editText, setEditText] = useState("");

  const addTodo = (e) => {
    e.preventDefault();
    const title = text.trim();
    if (!title) return;

    const newTodo = { id: Date.now(), title, done: false };
    setTodos((prev) => [newTodo, ...prev]);
    setText("");
  };

  const toggleTodo = (id) => {
    setTodos((prev) =>
      prev.map((t) => (t.id === id ? { ...t, done: !t.done } : t))
    );
  };

  const deleteTodo = (id) => {
    setTodos((prev) => prev.filter((t) => t.id !== id));
    // setEditingId(null);
  };
}
```

```
      <button type="button" onClick={() => saveEdit(t.id)}>
        Save
      </button>
      <button type="button" onClick={cancelEdit}>
        Cancel
      </button>
    </div>
  ) : (
    <div style={{ display: 'flex', gap: 10, alignItems: 'center' }}>
      <span
        onClick={() => toggleTodo(t.id)}
        style={{
          cursor: 'pointer',
          textDecoration: t.done ? 'line-through' : 'none',
        }}
        title="Click to toggle complete"
      >
        {t.title}
      </span>

      <button type="button" onClick={() => startEdit(t)}>
        Edit
      </button>

      <button type="button" onClick={() => deleteTodo(t.id)}>
        Delete
      </button>
    </div>
  )
</li>
)))
</ul>
</div>
);
}
```

3) How to Run

From the project folder:

[] Bash

```
npm install
npm run dev
```

Then open the local URL shown in the terminal (usually `http://localhost:5173/`).

4) Manual Test Steps

1. Type a task in the input and click **Add** → task appears in the list.
2. Click a task title → it toggles completed (line-through).
3. Click **Edit** → change text → click **Save** → title updates.
4. Click **Cancel** while editing → editing stops without saving.
5. Click **Delete** → task is removed.
6. Try adding an empty task → nothing should be added.

Version 4 — Layer: Quality Criteria

Prompt:

“Build a Todo app frontend in React with these features: add, list, toggle complete, edit title, delete. This is for a beginner portfolio project using Vite + React, with no backend (use local state only). Organize your answer into: (1) File/Component Structure, (2) Code, (3) How to Run, (4) Manual Test Steps.

Quality criteria: use semantic HTML, accessible form labels, keyboard-friendly controls, and mobile-first responsive layout.”

Representative Output:

- What changed in the prompt: Added quality criteria (a11y + semantic + responsive).
- What improved in the output: The output used semantic elements, connected labels with inputs, a real checkbox for keyboard-friendly task toggling, and responsive CSS for smaller screens.
- What still failed (honest): The output became longer because it included extra CSS and accessibility-related markup, which made it harder to read and paste quickly as a beginner.
- What I would try next: Add verification requirements (self-check list).

```
Version 4 Prompt:
Build a Todo app frontend in React with these features: add a task, list tasks,
toggle complete, edit a task title, and delete a task. This is for a beginner
portfolio project using Vite + React, with no backend (use local component state
only).
Organize your answer into exactly these sections:

File/Component Structure
Code
How to Run
Manual Test Steps
Quality criteria: use semantic HTML, accessible form labels, keyboard-friendly
controls, and a mobile-first responsive layout.
```

gpt-5.2-high

File/Component Structure

Keep it beginner-friendly:

```
{ } text

todo-prompt-ladder/
src/
  App.jsx
  index.css
  main.jsx
```

Code

```
src/App.jsx

{ } React

import { useId, useMemo, useState } from "react";

export default function App() {
  const inputId = useId();

  const [text, setText] = useState("");
  const [todos, setTodos] = useState([
    { id: 1, title: "Learn React basics", done: false },
    { id: 2, title: "Build a portfolio project", done: true },
  ]);

  const [editingId, setEditingId] = useState(null);
  const [editText, setEditText] = useState("");

  const remainingCount = useMemo(
    () => todos.filter((t) => !t.done).length,
    [todos]
  );

  const addTodo = (e) => {
    e.preventDefault();
    const title = editText.trim();
    if (!title) return;

    const newTodo = { id: Date.now(), title, done: false };
    setTodos((prev) => [...prev, newTodo]);
    setEditText("");
  };
}
```

src/index.css

() CSS

```
:root {
  color-scheme: light dark;
  --bg: #0b0f19;
  --card: rgba(255, 255, 255, 0.06);
  --text: rgba(255, 255, 255, 0.92);
  --muted: rgba(255, 255, 255, 0.65);
  --border: rgba(255, 255, 255, 0.16);
  --primary: #7c5cff;
  --danger: #ff5c7a;
}

* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
  background: radial-gradient(1200px 700px at 50% -200px, #2a1b6e 0%, var(--bg) 55%);
  color: var(--text);
}

.page {
  width: min(900px, 100%);
  margin: 0 auto;
  padding: 24px 16px 48px;
}

.header {
  display: flex;
  gap: 10px;
  align-items: center;
  cursor: pointer;
  user-select: none;
}

.toggle input { width: 18px; height: 18px; }
.done { text-decoration: line-through; color: var(--muted); }

.actions { display: flex; gap: 8px; align-items: center; }

.btn {
  border: 1px solid var(--border);
  background: rgba(255, 255, 255, 0.06);
  color: var(--text);
  padding: 9px 12px;
  border-radius: 10px;
  cursor: pointer;
}

.btn:hover { border-color: rgba(255, 255, 255, 0.32); }
.btn:focus { outline: 2px solid color-mix(in oklab, var(--primary), white 10%); outline-offset: 2px; }

.primary { background: color-mix(in oklab, var(--primary), black 25%); border-color: transparent; }
.danger { background: color-mix(in oklab, var(--danger), black 25%); border-color: transparent; }

.empty { margin: 0; color: var(--muted); }
@media (max-width: 520px) {
  .row { align-items: stretch; }
  .actions { width: 100%; justify-content: flex-end; }
}
```

"Note: Because this version adds quality criteria, the output is longer (CSS + more semantic/a11y-friendly markup)."

How to Run

From your project folder:

() Bash

```
npm install
npm run dev
```

Open the URL shown in the terminal (usually `http://localhost:5173/`).

If you paste the CSS, make sure `src/main.jsx` imports `./index.css` (Vite does by default).

Manual Test Steps

1. Type a task → press **Add** (or Enter) → task appears.
2. Use **Tab** to reach the checkbox → press **Space** → task toggles complete.
3. Click **Edit** → change text → click **Save** → title updates.
4. Click **Cancel** while editing → no changes saved.
5. Click **Delete** → task removed.
6. Resize window to mobile width → layout should still look clean and readable.

Version 5 — Layer: Verification Requirements

Prompt:

“Build a Todo app frontend in React with these features: add, list, toggle complete, edit title, delete. This is for a beginner portfolio project using Vite + React, with no backend (use local state only).

Organize your answer into: (1) File/Component Structure, (2) Code, (3) How to Run, (4) Manual Test Steps.

Quality criteria: semantic HTML, accessible labels, keyboard-friendly controls, mobile-first responsive layout.

At the end, add: (A) Assumptions you made, (B) a verification checklist with 6 bullets to confirm it works.”

Representative Output:

- What changed in the prompt: Added verification requirements (assumptions + checklist).
- What improved in the output: The output ended with assumptions and a six-item verification checklist, making the limitations and testing steps clear.
- What still failed: Often increases length (extra text). If too long, you’d make verification optional next time.
- What I would try next: Add constraints like “keep code under X lines” OR examples.

The screenshot shows a chat window with a dark theme. At the top, a light gray box contains the 'Version 5 Prompt'. Below it, the chat history shows a user message 'gpt-5.2-high' and an AI response. The AI response is divided into sections: 'File/Component Structure' and 'Code'. The 'File/Component Structure' section shows a directory tree for 'todo-prmpt-ladder/' with a subdirectory 'src/' containing 'App.jsx'. The 'Code' section shows the content of 'src/App.jsx' with React code for a todo app.

```
Version 5 Prompt:
Build a Todo app frontend in React with these features: add a task, list tasks,
toggle complete, edit a task title, and delete a task. This is for a beginner
portfolio project using Vite + React, with no backend (use local component state
only).
Organize your answer into exactly these sections:
File/Component Structure
Code
How to Run
Manual Test Steps
Quality criteria: use semantic HTML, accessible form labels, keyboard-friendly
controls, and a mobile-first responsive layout.
At the very end add:

Assumptions (3 bullets)
Verification Checklist (6 checkboxes)
```

gpt-5.2-high

File/Component Structure

Beginner-friendly structure:

```
{ } text

todo-prmpt-ladder/
  src/
    App.jsx
```

Code

src/App.jsx

```
[ ] React

import { useState, useId, useText, useText } from "react";

export default function App() {
  const addInputId = useId();

  const [text, setText] = useState("");
  const [todos, setTodos] = useState([
    { id: 1, title: "Learn React basics", done: false },
    { id: 2, title: "Build a portfolio project", done: true },
  ]);

  const [editingId, setEditingId] = useState(null);
  const [editText, setEditText] = useState("");

  const remainingCount = useText(
    () => todos.filter((t) => !t.done).length,
    [todos]
  );

  const addTodo = (e) => {
    e.preventDefault();
    const title = text.trim();
    if (!title) return;

    setTodos((prev) => [...prev, { id: Date.now(), title, done: false }]);
  };
}
```

src/index.css

{ } CSS

```
:root {
  color-scheme: light dark;
  --bg: #0b0f19;
  --card: rgba(255, 255, 255, 0.06);
  --text: rgba(255, 255, 255, 0.92);
  --muted: rgba(255, 255, 255, 0.65);
  --border: rgba(255, 255, 255, 0.16);
  --primary: #7c5cff;
  --danger: #ff5c7a;
}

* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
  background: radial-gradient(1200px 700px at 50% -200px, #2a1b6e 0%, var(--bg) 55%);
  color: var(--text);
}

.page {
  width: min(900px, 100%);
  margin: 0 auto;
  padding: 24px 16px 48px;
}

.header {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  align-items: baseline;
```

How to Run

From the project folder:

{ } Bash

```
npm install
npm run dev
```

Open the URL shown in the terminal (usually `http://localhost:5173/`).

Manual Test Steps

1. Add a task → click **Add** (or press **Enter**) → task appears.
2. Use **Tab** to focus a checkbox → press **Space** → toggles complete.
3. Click **Edit** → change text → click **Save** → updates title.
4. Click **Cancel** during edit → reverts to view mode.
5. Click **Delete** → removes the task.
6. Resize to mobile width → layout should remain readable and buttons wrap nicely.

Assumptions (3 bullets)

- You are using a fresh **Vite + React** project and `src/main.jsx` imports `./index.css` .
- This project is **frontend-only** (no database/API), so todos reset on page refresh.
- IDs generated via `Date.now()` are "good enough" for a beginner demo (not for production).

Verification Checklist (6 checkboxes)

- ☐ I can add a todo using the input + **Add** button (and **Enter** works).
- ☐ I can toggle complete using the **checkbox** (keyboard: **Tab** + **Space** works).
- ☐ I can edit a todo title and **Save** updates it.
- ☐ **Cancel** exits edit mode without changing the title.
- ☐ **Delete** removes a todo (and doesn't break edit mode).
- ☐ On mobile width (≤520px), layout wraps and remains usable/readable.

Final Reusable Prompt (Frontend — React + Vite)

Prompt:

Build a Todo app frontend in React for a beginner portfolio project using Vite + React. This is frontend-only (no backend/API). Use local component state only.

Required features:

Add a task, List tasks, Toggle complete/incomplete, Edit a task title (Save + Cancel), Delete a task.

Organize your response into exactly these sections:

File/Component Structure, Code, How to Run, Manual Test Steps.

Quality criteria:

Use semantic HTML (main/header/section where appropriate), Use accessible form labels (label + htmlFor/id), Controls must be keyboard-friendly (Tab navigation; toggling should work without relying on click-only), Mobile first responsive layout (should remain usable on small screens).

At the very end, include:

Assumptions (exactly 3 bullet points), Verification Checklist (exactly 6 checkboxes).

Important constraints:

Keep it beginner-friendly (clear variable names, short comments only where necessary), Do not use external UI libraries (no Tailwind/Bootstrap/MUI) unless explicitly asked, Do not add backend code or API calls.

